Lesson notes · for you

How work grows: big O

Programming · Level 4 · Lesson 1

Time
20 min

Materials
A phone with this lesson downloaded; A notebook for the step counts

Learning goals

- Count the steps an algorithm takes as the input grows.
- Classify code as O(1), O(log n), O(n), O(n log n) or $O(n^2)$.
- Replace an $O(n^2)$ method with a faster one.

Activity: Every pair, or one pass?

1. Warm-up (3 min): Write down a guess: how long would comparing every pair of a million records take on a phone?
2. Count (7 min): With paper, count the comparisons every-pair needs for 4, 8 and 16 items. What happens each time n doubles?
3. Improve (7 min): Write the set version and count its checks for the same sizes.
4. Explain (3 min): Write both as $O(n^2)$ and O(n), then check your warm-up guess with ten million steps a second.

Counting on paper is enough: the pattern appears by 16 items.

Discussion

- Why do computer scientists count steps instead of seconds?
- Where have you seen a hidden loop?
- When is $O(n^2)$ good enough?

Check yourself

- Write a loop that is O(n), and one that is $O(n^2)$, for the same list.

Try it this week
Find a shared birthday among ten friends two ways: compare every pair, then go round once, ticking each birthday on a year calendar with one box per day: a box already ticked is a shared birthday. Count the comparisons (or looks) each way.

Worksheet

How work grows: big O

Programming · Level 4 · Lesson 1

Name _____ Class _____ Date _____

1 **A program loops over n names. Which line, inside that loop, hides a second loop?**

◯ count = count + 1 ◯ if name in seen: (seen is a set) ◯ if name in names: (names is a list)

2 **Write first_repeat(values): it returns the first value that appears a second time, or None. It must stay fast for 20,000 values, so use a set, not a loop inside a loop.**

```
def first_repeat(values):
    # return the first value seen twice, or None
    # (a set keeps it fast: one pass)
    return None
```

Write your program:

3 **What is the big O of this function, for a list of n values?**

```
def has_pair(a, target):
    for x in a:
        for y in a:
            if x + y == target:
                return True
    return False
```

◯ $O(n^2)$ ◯ O(n) ◯ O(1) ◯ O(log n)

4 **Fill the gaps: the program counts how many times 1,000 can be halved before it reaches 1.**

```
n = 1000
steps = 0
while n _____ 1:
    n = n _____ 2
    steps = steps + 1
print(steps)
```

Word bank: `<` `-` `>` `//`

5 **Match each piece of code to its big O.**

Read a[0]	• • • •	O(n log n)
Binary search on a sorted list	• • • •	O(n)
One loop over the list	• • • •	O(1)
Merge sort	• • • •	$O(n^2)$
A loop inside a loop over the list	• • • •	O(log n)

Find a program you wrote with a loop inside a loop. What is its big O? Could a set or a dictionary remove the inner loop?

Check your answers

How work grows: big O

Programming · Level 4 · Lesson 1

1 **if name in names: (names is a list)**

Checking with in on a list looks at the items one by one, so inside a loop over n names it makes O(n²) work.

2

```
def first_repeat(values):
    seen = set()
    for v in values:
        if v in seen:
            return v
        seen.add(v)
    return None
```

One pass, with a set to remember what you have seen: O(n).

3 **O(n²)**

A loop inside a loop over the same list: n × n pairs in the worst case, O(n²).

4

```
n = 1000
steps = 0
while n > 1:
    n = n // 2
    steps = steps + 1
print(steps)
```

Halving 1,000 reaches 1 in 9 steps (500, 250, 125, 62, 31, 15, 7, 3, 1): that is $\log_2 n$, the growth of binary search.

5

Read a[0] — O(1)

Binary search on a sorted list — O(log n)

One loop over the list — O(n)

Merge sort — O(n log n)

A loop inside a loop over the list — O(n²)

One step, halving, one pass, split-and-merge, every pair: the five growth classes you will meet most.